Quantitative Evaluation of Kernel Fusion Strategies for Transformer Inference

Sankirth Kalahasti, Rishabh Budhouliya, Aadit Fadia
Tandon School of Engineering
New York University
New York, USA
{sk11617, rb5727, aaf9407}@nyu.edu

***Abstract*—Transformer inference performance is increasingly dominated by memory traffic and kernel launch overhead rather than raw computation. While fused kernels such as FlashAttention demonstrate large speedups, the individual effects of fusion granularity, scheduling, and memory reuse remain poorly isolated. This project quantitatively evaluates these dimensions on a GPT-2 Small transformer block using an NVIDIA A100 GPU. We report three key observations: (1) naively implemented fused kernels can incur severe performance regressions (up to 30×) due to bandwidth underutilization, demonstrating that fusion without careful memory scheduling can be counterproductive; (2) in-kernel software pipelining exhibits implementation-sensitive behavior, where shallow prefetching may degrade performance while deeper pipelines (4–5 stages) improve throughput by up to 8.7% for memory-bound kernels; and (3) manual tiling strategies are highly sensitive to register pressure and tile geometry, with poorly chosen configurations reducing occupancy, while well-tuned tiling delivers substantial gains over compiler-managed baselines. Overall, our results indicate that for small-scale transformer blocks, carefully scoped fusion combined with kernel-specific scheduling outperforms overly complex megakernel designs.**

***Index Terms*—GPU Optimization, Kernel Fusion, Triton, CUDA, Transformers, GPT-2, Software Pipelining.**

I. Introduction

A. Background and Motivation

The deployment of Large Language Models (LLMs) is increasingly constrained by inference latency. On modern accelerators such as the NVIDIA A100, performance is often limited not by arithmetic throughput (FLOPS), but by memory bandwidth and kernel launch overhead. In standard PyTorch implementations, transformer blocks execute as sequences of discrete kernels, leading to repeated global memory traffic between operations such as LayerNorm, linear projections, and activation functions. This execution model motivates kernel fusion as a means to reduce launch overhead and improve data locality, but also introduces new challenges related to resource usage and scheduling.

B. Problem Statement

While fused kernels are known to improve performance in many settings, it remains unclear which optimization dimensions—specifically fusion granularity, instruction scheduling, and memory reuse—most strongly influence transformer inference performance in practice. Reducing kernel launch overhead alone does not guarantee improved throughput, as performance depends on complex interactions between resource utilization, memory access patterns, and execution scheduling within a kernel. As a result, it is difficult to reason a priori about when deeper fusion, more aggressive scheduling, or manual memory reuse will be beneficial, and when these optimizations may instead introduce performance regressions.

C. Objectives and Scope

This project aims to isolate and evaluate three orthogonal axes of optimization on a GPT-2 Small transformer block:

- **Track A (Fusion Granularity):** Does deeper fusion, characterized by fewer kernel launches, necessarily translate to faster execution?
- **Track B (In-Kernel Scheduling):** How does the depth of software pipelining (num_stages) influence memory latency hiding and overall throughput?
- **Track C (Memory Reuse):** Can explicit memory reuse through manual shared-memory buffering and prefetching in CUDA and Triton outperform hardware-managed caching mechanisms?

II. Methodology

A. Experimental Setup

We optimized a GPT-2 Small transformer block ($d_{\text{model}} = 768$, $N_{\text{head}} = 12$). All experiments were conducted on an NVIDIA A100-SXM4 GPU (40 GB HBM2, 108 SMs). The workload was executed in evaluation mode to measure pure inference performance using synthetic inputs, with batch sizes ranging from 8 to 64 and sequence lengths from 512 to 1024.

Performance was measured using wall-clock kernel latency and achieved memory bandwidth. NVIDIA Nsight Systems (`nsys`) was used for profiling, capturing kernel timelines, NVTX ranges, and compiler-reported statistics such as register usage and occupancy.

B. Optimization Tracks

1) Track A: Fusion Granularity: We implemented three distinct levels of fusion to test launch overhead hypotheses:

- **A0 (Baseline):** Unfused PyTorch (∼8 launches/block).
- **A1 (Block-Level):** Fused LN+QKV, Softmax+Attn, LN+MLP (∼4 launches).
- **A2 (Megakernel):** Full layer fusion including residuals (∼3 launches).

2) Track B: In-Kernel Scheduling: Using the Triton compiler, we manipulated the `num_stages` parameter to control the depth of software pipelining for global memory loads:

- **B0 (Sequential):** 1 stage (Synchronous loads).
- **B1 (Moderate):** 2-3 stages (Moderate prefetching).
- **B2 (Aggressive):** 4-5 stages (Deep prefetching).

3) Track C: Memory Reuse: Track C evaluates the impact of explicit, programmer-managed memory reuse compared to compiler- and hardware-managed caching. We progressively increased the degree of manual control over memory placement, buffering, and reuse across Triton and CUDA implementations:

- **C0 (Compiler-Managed Baseline):** Standard Triton implementations of FlashAttention and MLP layers, relying on hardware-managed caches and compiler-selected memory reuse.
- **C1 (Prefetching and Double Buffering):** Explicit prefetching of weights and activations into shared memory, combined with double buffering to overlap global memory loads with computation.
- **C2 (Aggressive Tiling and Multi-Buffering):** Fine-grained tiling with "ping-pong" shared-memory buffers for model weights and intermediate activations, designed to maximize data reuse at the cost of increased register and shared-memory usage.

III. Experimental Results

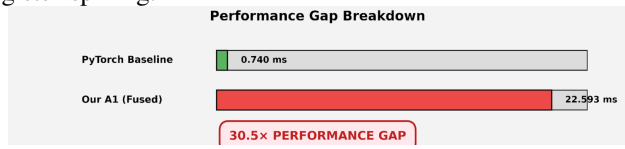
A. Track A: The "Fusion Fallacy"

Contrary to the hypothesis that fewer kernel launches yield higher performance, our results demonstrated a non-linear degradation with deeper fusion.

Bandwidth Collapse:

- **A0 (Baseline):** 0.73 ms Latency — 48.2 GB/s Bandwidth.
- **A1 (Fused):** 22.59 ms Latency — 1.3 GB/s Bandwidth.

This represents a **30.5x slowdown**. Despite reducing kernel launches by 50%, the effective memory bandwidth collapsed by 97%. This was attributed to suboptimal memory coalescing in the custom kernels compared to the highly optimized cuBLAS calls used by PyTorch. The "Megakernel" (A2) further degraded performance (23.31 ms) due to excessive register spilling.



B. Track B: The Pipelining Trough

Track B revealed a surprising non-monotonic relationship between pipeline depth and latency, which we term the "Pipelining Trough."

Anomaly 1: The Trough. Moderate pipelining (B1) consistently performed *worse* than sequential execution (B0). The overhead of managing the software pipeline (prologue/epilogue instructions) outweighed the benefits of prefetching until the pipeline was deep enough (B2) to fully hide the global memory latency.

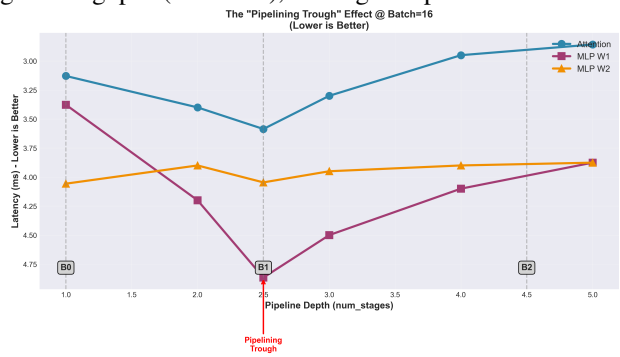
TABLE I
Track B Performance Summary (Batch=16)

Metric	B0 (Seq)	B1 (Pipe)	B2 (Aggr)
Latency (ms)	12.66	14.31	**12.35**
Throughput (Tok/s)	5.18M	4.58M	**5.31M**
Speedup vs Base	1.67x	1.48x	**1.72x**

Anomaly 2: Kernel-Specific Optimization.

- **Attention (Memory-Bound):** Benefited from Aggressive Pipelining. B2 was 8.7% faster than B0. The streaming nature of K/V tile loading is ideal for deep prefetching.
- **MLP W1 (Compute-Bound):** Suffered from Pipelining. B0 was 13% *faster* than B2. The high arithmetic intensity meant that instruction fetch overhead for pipelining stalled the tensor cores.

Anomaly 3: The Batch Size 1 Reversal. At Batch Size = 1, the behavior of the MLP W1 kernel inverted. B2 became 17% faster than B0. This suggests that when the grid size is small, the latency hiding benefits of pipelining dominate, whereas, at high throughput (Batch 16), the register pressure dominates.



C. Track C: Complexity vs. Efficiency

Track C evaluates the tradeoff between explicit, programmer-managed memory reuse and its impact on kernel efficiency, memory bandwidth utilization, and occupancy. Table II provides a quantitative summary of latency, throughput, and achieved bandwidth across all C variants, while Figures 1 and 2 illustrate scaling behavior.

a) Latency Analysis: Across all batch sizes and sequence lengths, **C0 consistently achieves the lowest latency**, making it the fastest overall configuration. The compiler-managed Triton baseline benefits from low register pressure and short kernel runtimes, enabling efficient SM utilization.

C1 (Triton) closely tracks C0 in latency and scales predictably as sequence length increases. Although C1 introduces additional buffering and prefetching logic, its overhead remains modest, indicating that moderate explicit memory reuse can be incorporated without significantly degrading execution efficiency.

In contrast, **C2 (Triton)** exhibits higher latency, with performance degrading as sequence length grows. This behavior

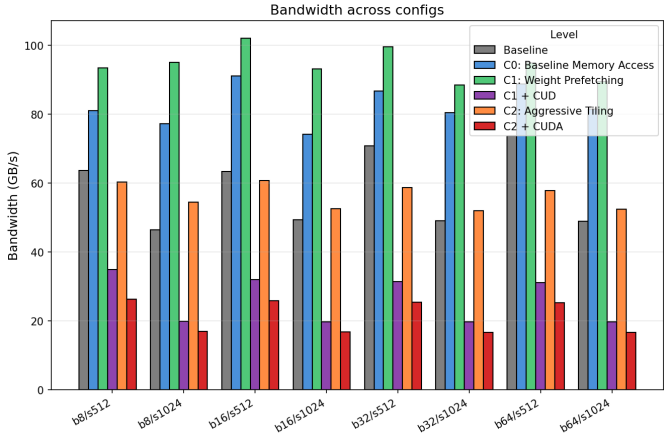


Fig. 1. Achieved memory bandwidth for Track C variants.

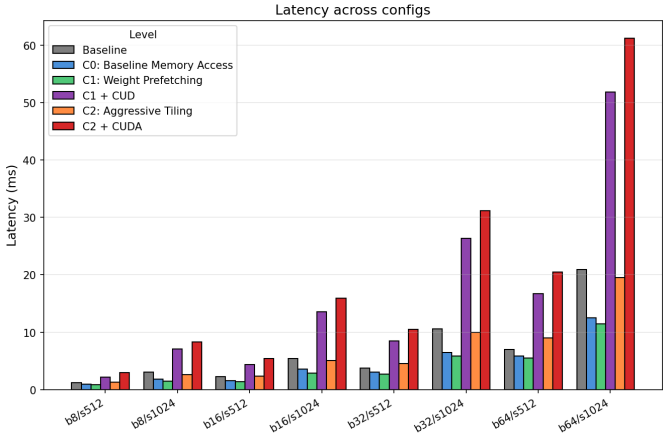


Fig. 2. End-to-end latency across batch sizes and sequence lengths for Track C variants.

reflects the increased cost of aggressive tiling and multi-buffering, which lengthens kernel execution time and limits scalability.

The **CUDA implementations of C1 and C2 are consistently the slowest**, particularly at larger batch sizes and sequence lengths. These kernels suffer from longer execution times and reduced throughput, highlighting the difficulty of manually implementing efficient memory reuse and synchronization.

b) Bandwidth Utilization: Effective memory bandwidth further explains the observed latency trends. **C1 (Triton) achieves the highest sustained bandwidth**, consistently outperforming all other variants. This indicates that moderate prefetching and double buffering improve memory throughput without overwhelming hardware resources.

C0 maintains strong but slightly lower bandwidth, reflecting efficient cache utilization without explicit reuse. While it does not reach C1's peak bandwidth, its simpler execution model enables it to remain latency-optimal.

C2 operates at moderate bandwidth despite its aggressive reuse strategy. Increased register and shared-memory usage reduce occupancy, preventing full utilization of available memory bandwidth.

The **CUDA variants of C1 and C2 exhibit the lowest bandwidth**, indicating poor memory utilization and ineffective latency hiding, particularly under larger workloads.

c) Kernel Efficiency: Kernel-level analysis shows that **C0 kernels are the shortest in duration**, directly contributing to their low end-to-end latency. **C1 kernels are slightly longer but remain stable** across workloads, consistent with their predictable scaling behavior.

C2 (Triton) kernels occupy an intermediate position, longer than C0 and C1 but shorter than the CUDA variants. The additional tiling and buffering logic increases instruction count and execution time.

The **CUDA C1/C2 kernels are significantly longer**, especially at larger batch sizes, reflecting increased synchronization overhead and inefficient scheduling.

TABLE II
PERFORMANCE SUMMARY FOR TRACK C VARIANTS.

Variant	Latency (ms)	Speedup vs. Baseline	Speedup vs. C0	Tokens/s	Bandwidth (GB/s)
Baseline	20.91	1.00	0.60	3.13M	48.82
C0	12.53	1.67	1.00	5.23M	81.48
C1	11.42	1.83	1.10	5.74M	89.41
C2	19.48	1.07	0.64	3.36M	52.40

d) Arithmetic Intensity: All Track C variants perform identical arithmetic operations and therefore exhibit identical arithmetic intensity. Consequently, runtime differences arise solely from implementation quality, memory behavior, and kernel efficiency rather than differences in computational workload.

IV. DISCUSSION

A. Interpretation of Results

a) What Determines Performance in Practice: Across all three tracks, performance was governed primarily by memory behavior and resource utilization rather than raw arithmetic throughput. Reducing kernel launch overhead through fusion was beneficial only when memory access patterns remained efficient. Aggressive fusion or reuse strategies that increased register pressure or reduced occupancy frequently negated their intended benefits, underscoring that memory bandwidth and latency hiding dominate inference performance for GPT-2 Small.

b) Track A: Fusion without Scheduling Is Insufficient: The unfused PyTorch baseline suffered from repeated global memory traffic and high kernel launch overhead, resulting in the lowest bandwidth utilization and Tensor Core efficiency. Moderate fusion (C0) reduced launch overhead while keeping register usage low, allowing high occupancy and stable performance across configurations. In contrast, deeper fusion without corresponding attention to scheduling and memory reuse led to bandwidth collapse, demonstrating that fusion depth alone is not a reliable predictor of performance.

c) Track B: Kernel-Dependent Scheduling Trade-offs: Track B revealed a clear *pipelining trough*, where moderate software pipelining (B1, `num_stages`=2–3) underperformed

both sequential execution (B0) and aggressive pipelining (B2). Importantly, no single scheduling strategy was universally optimal. Attention kernels, which are memory-bound with streaming access patterns, benefited from aggressive pipelining, achieving up to 8.7% speedup with B2. In contrast, the first MLP projection (W1), which is more compute-bound with high arithmetic intensity, performed best without pipelining, while the second projection (W2), which is memory-bound, consistently favored deeper pipelining. These results highlight that scheduling decisions must be made at the kernel level and validated under the target workload configuration.

d) Track C: Balancing Reuse and Resource Pressure: Track C illustrated the trade-off between explicit memory reuse and kernel efficiency. The C0 variant, which relied on compiler-managed caching while keeping intermediates on-chip through fusion, achieved the best overall latency and stability due to low register pressure and high occupancy. C1 introduced double buffering and prefetching, which in Triton improved effective bandwidth and modestly hid memory latency, but in CUDA incurred additional control and synchronization overhead. C2 pursued aggressive tiling and multi-buffering, but the resulting register pressure and shared-memory usage caused occupancy collapse and longer kernel runtimes, particularly for GPT-2 Small. These findings indicate that reuse-focused optimizations must be carefully balanced against their resource costs.

e) Fusion Is Ineffective for Compute-Bound Kernels: Our results indicate that kernel fusion provides limited benefit for compute-bound layers and can, in some cases, degrade performance. When arithmetic intensity is high and Tensor Core utilization is already near saturation, reducing kernel launch overhead or intermediate memory traffic yields marginal gains. In these cases, additional fusion increases register pressure, instruction complexity, and scheduling constraints without alleviating the primary bottleneck. This often leads to reduced occupancy and longer kernel runtimes, making aggressive fusion counterproductive for compute-bound kernels.

B. Comparison with Previous Studies

Prior work such as FlashAttention demonstrates that carefully designed fused kernels with structured tiling and scheduling can achieve large speedups for attention workloads. Our results are consistent with these findings, but also show that similar strategies do not automatically generalize to all components of a transformer block or to smaller model sizes. In particular, we find that moderate fusion combined with kernel-specific scheduling often outperforms more complex megakernel designs. This complements existing literature by highlighting that the effectiveness of fusion, pipelining, and reuse depends strongly on kernel characteristics, model scale, and hardware resource constraints.

C. Challenges and Limitations

a) Triton-Specific Challenges: Triton provides powerful abstractions for kernel fusion and scheduling, but limits explicit control over shared memory, causing tile reuse to depend on L1/L2 behavior rather than guaranteed residency. Designing reusable tiles using registers alone proved difficult without triggering register spills, and prefetch distance tuning was fragile due to tight register limits. Reuse-focused tiling strategies frequently reduced occupancy, and cache behavior could not be directly observed, requiring reuse effects to be inferred from timing and bandwidth metrics. Additionally, the Triton autotuner occasionally selected tile shapes that conflicted with reuse and prefetching patterns.

b) CUDA-Specific Challenges: Manual CUDA implementations introduced a different set of difficulties. Designing multi-page shared-memory buffers required precise layout and synchronization, and coordinating prefetch, reuse, and computation across warps was error-prone. Tensor Core programming via WMMA imposed rigid tile shapes and alignment constraints, limiting flexibility. High shared-memory and register usage further constrained occupancy. Debugging subtle kernel issues such as NaNs and silent race conditions was challenging, and the PyTorch extension workflow introduced additional complexity through versioning and ABI compatibility. Overall, the learning curve for achieving competitive performance in CUDA was substantially steeper than in Triton.

c) General Limitations: A key limitation of this study was the lack of access to NVIDIA Nsight Compute, which prevented direct inspection of instruction-level behavior such as register spills and memory dependency stalls. Furthermore, tile sizes and scheduling parameters were manually selected rather than auto-tuned. Future work could explore automated tuning of tile geometry and pipeline depth, particularly with respect to the A100's cache hierarchy, to further clarify the performance boundaries observed in this study.tentially mitigate the bandwidth collapse seen in Track A.

V. Conclusion

This study examined the performance trade-offs of kernel fusion, in-kernel scheduling, and explicit memory reuse for transformer inference. By isolating these optimization axes, we show that performance is governed less by the presence of fusion itself and more by how fusion interacts with memory behavior, resource usage, and kernel-specific characteristics. While absolute performance varies with model size and hardware, the underlying trade-offs observed in this work persist across transformer workloads. Our key conclusions are as follows:

1) **Fusion Must Be Scoped and Memory-Aware:** Moderate, block-level fusion that reduces launch overhead while preserving efficient memory access patterns consistently outperformed deeper megakernel designs in our experiments. Aggressive fusion without corresponding attention to scheduling and resource constraints often led to bandwidth underutilization and reduced occupancy.
2) **Scheduling Is Kernel-Dependent:** Software pipelining exhibited no universal optimum. Memory-bound kernels benefited from deeper pipelining that improved memory latency hiding, while compute-bound kernels often performed best with minimal or no pipelining. Effective

scheduling decisions therefore require per-kernel profiling under the target workload configuration.

3) **Explicit Reuse Has Diminishing Returns:** While moderate prefetching and buffering can improve effective bandwidth, aggressive manual memory reuse frequently increased register and shared-memory pressure, leading to occupancy collapse and longer kernel runtimes. Compiler-managed approaches were generally more robust in navigating these trade-offs across evaluated configurations.

Overall, our results suggest that effective transformer inference optimization requires balancing fusion granularity, scheduling depth, and memory reuse rather than aggressively maximizing any single dimension. These insights provide guidance for designing scalable inference kernels across a wide range of transformer model sizes and deployment settings.

REFERENCES

[1] S. Kalahasti, R. Budhouliya, A. Fadia, "GPT2 Kernel Fusion," NYU HPML Project, 2025.

[2] NVIDIA, "NVIDIA A100 Tensor Core GPU Architecture," Whitepaper.

[3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems*, 2022.

[4] P. Tillet, H. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 2019 MAPL*, 2019.

[5] Triton Tutorials, "Getting Started Tutorials," Available: https://triton-lang.org/main/getting-started/tutorials/index.html